A Full-Digital Heterogeneous Computing-in-Memory With Dual-Bit Serialized Weight Stream and Ping-Pong Computing for Efficient Transformer Acceleration

Dengfeng Wang, *Student Member, IEEE*, Liukai Xu, Shuai Yuan, Weifeng He, *Senior Member, IEEE*, Guanghui He, *Member, IEEE*, Fengbin Tu, *Senior Member, IEEE*, Xueqing Li, *Senior Member, IEEE*, and Yanan Sun, *Senior Member, IEEE*

***Abstract*—Vision Transformers (ViTs) have demonstrated remarkable performance across various computer vision tasks. However, the self-attention mechanism in transformers incurs high processing loads and costly off-chip data movement in conventional von Neumann architectures. This paper proposes a full-digital heterogeneous Computing-in-Memory (CIM) *in-situ* hybridizing differential RRAM and SRAM to enable efficient on-chip weight storage and flexible row-column dual-directional access for efficient transformer acceleration. A Ping-Pong computing dataflow is introduced to maximize the throughput density by allocating static and dynamic computations across CIM macros. A low-toggle weight tuning algorithm is proposed to minimize the dynamic power under the dual-bit-serial CIM scheme by introducing only small weight rounding distance. Besides, a patch-channel skip mechanism is used to reduce the redundant computing for dynamic matrix with high sparsity. Experimental results on different transformer models show that the proposed DBS-CIM architecture reduces layer-wise computing energy by up to 59.9% compared to conventional digital CIM. Compared to prior homogeneous CIM designs, it achieves 2.0× to 3.3× higher end-to-end energy efficiency, and outperforms existing heterogeneous CIM designs by 2.13× in computational density.**

***Index Terms*—Heterogeneous computing-in memory, transformer, attention, digital accelerator.**

I. Introduction

TRANSFORMERS [1] have demonstrated remarkable performance across a wide range of computer vision tasks [2], [3], [4]. The attention mechanism in transformer models offers the capability of capturing long-range dependencies within an image and is flexible and scalable with deeper model depth. The computing flows of two mainstream attention mechanisms are shown in Fig. 1(a). For non-linear attention (NLA) model, the input tokens are first multiplied by static weight matrixes W_q, W_k, and W_v to obtain the dynamic weights Q, K, and V. The attention score P is computed via the dot product of Q and K^T, followed by a softmax activation. The attention output S is then derived from V and P. Although providing good global and local feature extraction capability, the NLA mechanism with softmax in transformers faces a computational complexity that grows quadratically with the number of input tokens in high resolution images. To mitigate this, the linear attention (LA) model has been introduced to reduce the computing complexity to linearity [3], [4]. This is achieved by applying a decomposable kernel function (e.g., ReLU) to Q and K, and strategically reordering the computation steps. The removal of softmax in linear attention compromises local feature extraction, which introduces a need for high-precision convolution layers [5]. Concurrently, accelerating transformers on conventional von Neumann computing hardware is hindered by high computational load [6], costly off-chip data movement [7], [8], and strict precision demands [9].

Computing-in-memory (CIM) has emerged as a promising approach [10], [11] to accelerate transformer inference by minimizing data movement. Most CIM accelerators are based on either resistive random-access memory (RRAM-CIM) [12],

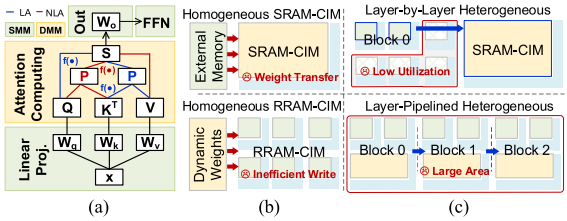


Fig. 1. (a) The computing flows of transformer models with different attention mechanisms and the challenges of previous (b) homogeneous [12], [13], [14], [15], [16], [17], [18], [19], [20], [21], [22] and (c) heterogeneous [24], [25], [26], [27], [28], [29], [30], [31], [32], [33] CIM methods to accelerate attention computing.

Received 31 January 2026; revised 18 April 2026 and 4 May 2026; accepted 17 May 2026. Date of publication 27 May 2026; date of current version 31 August 2026. This work was supported in part by the Smart Grid-National Science and Technology Major Project under Grant 2024ZD0802103, in part by the National Natural Science Foundation of China under Grant 92573106 and Grant U24B6015, and in part by Shanghai 2024 "Science and Technology Innovation Action Plan" Fundamental Research Program in Integrated Circuit under Grant 24JD1400300. This article was recommended by Associate Editor S. Taneja. *(Corresponding author: Yanan Sun.)*

Dengfeng Wang, Liukai Xu, Shuai Yuan, Weifeng He, Guanghui He, and Yanan Sun are with the School of Integrated Circuits, Shanghai Jiao Tong University, Shanghai 200240, China (e-mail: yungsk@sjtu.edu.cn; xuliukai@sjtu.edu.cn; yuanshuai_1994@sjtu.edu.cn; hewf@sjtu.edu.cn; guanghui.he@sjtu.edu.cn; sunyanan@sjtu.edu.cn).

Fengbin Tu is with the Department of Electronic and Computer Engineering, The Hong Kong University of Science and Technology, Hong Kong, China (e-mail: fengbintu@ust.hk).

Xueqing Li is with the Department of Electronic Engineering, Tsinghua University, Beijing 100084, China (e-mail: xueqingli@tsinghua.edu.cn).

Digital Object Identifier 10.1109/TCSI.2026.3695668

1549-8328 © 2026 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:26:46 UTC from IEEE Xplore. Restrictions apply.

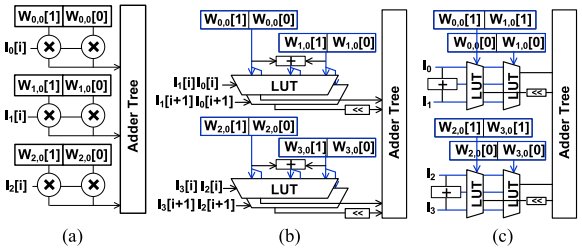


Fig. 2. Previous digital CIM methods. (a) Input-serial-weight-parallel with no hardware reuse [34]. (b) Input-parallel-weight-parallel with weight reuse [35]. (c) Input-parallel-weight-parallel with input reuse [36].

[13], [14], [15], [16] or static random-access memory (SRAM-CIM) [17], [18], [19], [20], [21], [22]. RRAM offers high on-chip storage capacity, thereby reducing off-chip DRAM access. SRAM-CIM, on the other hand, provides faster and more energy-efficient write operations. However, as shown in Fig. 1(b), neither RRAM nor SRAM alone is ideally suited for attention computation. RRAM suffers from low endurance, high latency, and high energy consumption in write operations [10], [23], making it inefficient for handling dynamic matrix computations in attention. Although SRAM supports dynamic computation effectively, its large cell area limits on-chip weight storage capacity [24], often necessitating off-chip DRAM access for weight loading, which degrades overall energy efficiency.

To overcome the limitations of homogeneous CIM and support parameter updating needs in state-of-the-art AI algorithm, recent heterogeneous CIM designs combining RRAM and SRAM have been proposed for efficient acceleration [24], [25], [26], [27], [28], [29], [30], [31], [32], [33]. Typically, recent works [25], [26], [27] use RRAM-CIM for fixed static matrix multiplications (SMM) and employ SRAM-CIM to handle dynamic matrix multiplications (DMM) in attention mechanism. However, as illustrated in Fig. 1(c), these approaches naïvely integrate isolated hybrid CIM arrays, facing their own challenges. A layer-by-layer heterogeneous design [25], [27], [31], [32] targeting layer-wise execution requires integrating only enough SRAM resources for single-layer dynamic computation around RRAM, leading to low hardware utilization of RRAM-CIM outside the current layer. In contrast, the layer-pipelined heterogeneous design [8], [30], [33] enables concurrent execution across multiple layers, which improves hardware utilization but incurs higher area overhead due to the need to store all dynamic and static weights. Thus, neither homogeneous nor conventional isolated heterogeneous CIM designs have yet provided an efficient solution for attention computation in transformers.

To efficiently execute the matrix multiplication task in transformers, CIM mechanisms in analog and digital domains have been widely explored. Analog CIM [21], [22] leverages the analog characteristics of memory circuits to perform multiply-and-accumulate (MAC) operations through current aggregation [21] or charge sharing [22], achieving high energy efficiency with simplified cell structures. However, it is vulnerable to process-voltage-temperature (PVT) variations with low computational precision and incurs high area and power overhead owing to analog-to-digital converters (ADCs). In contrast, digital CIM [34], [35], [36] has gained increasing

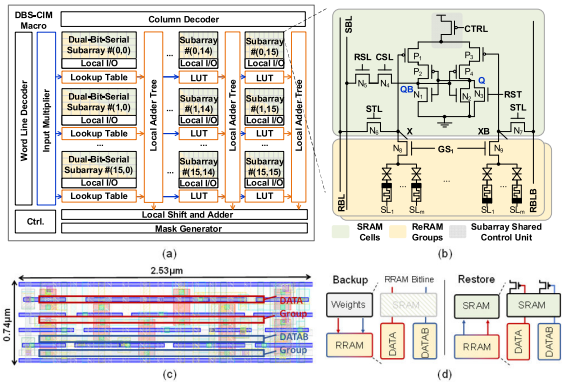


Fig. 3. (a) The proposed dual-bit-serial CIM (DBS-CIM) macro. (b) The schematic of proposed DBS-CIM cell. (c) The layout of proposed DBS-CIM cell. (d) The primary operations supported by the proposed DBS-CIM for enabling on-chip inference.

attention recently, which employs dedicated digital computing circuits in memory that support higher precision [20] for robust transformer inference. Previous digital CIM schemes are summarized in Fig. 2. The first type of digital bit-serial CIM (BS-CIM) embeds bit-wise multipliers within the memory subarray [34]. Inputs are processed in a bit-serial manner, while weights are read in parallel. Partial sums are accumulated using adder trees within the array. This approach, however, suffers from high area and power consumption due to adder trees. The second type of weight-reuse CIM (WR-CIM) uses pre-computed lookup tables (LUTs) within array to perform partial summations for weights sharing the same output channel [35]. By reusing pre-computed results, this method reduces adder tree overhead, but it does not exploit reuse opportunities across output channels. The third type of input-reuse CIM (IR-CIM) precomputes input-related results and broadcasts them to multiple output channels [36]. While this improves hardware reuse, the parallelly processed weights and inputs lead to unflexible data access and high toggle rate due to low bit correlation between different weights.

As summarized in Table I, none of these existing digital CIM designs offers low hardware complexity, high flexibility for data access, and low computational power consumption simultaneously for transformer acceleration. To address the above challenges, a fully digital heterogeneous CIM architecture based on *in-situ* hybridization of RRAM and SRAM is proposed in this work for efficient transformer acceleration. Specifically, by *in-situ* hybridizing SRAM with differential RRAM storage, the proposed hybrid CIM cell enables on-chip storage of all transformer weights while maintaining a high data restore yield. A Ping-Pong computing flow is also proposed for efficiently allocating the DMM and SMM tasks in CIM arrays to significantly improve the hardware utilization in previous heterogeneous CIM architecture. Additionally, an input-parallel weight-serial digital CIM scheme with a low-toggle tuning algorithm is employed to improve the energy efficiency of CIM while supporting flexible skipping mechanisms. The main contributions are listed as follows:

- A high-density hybrid CIM cell with differential RRAM storage is proposed to ensure high restore yield and flexible data access with row-and-column-wise selection.

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:26:46 UTC from IEEE Xplore. Restrictions apply.

TABLE I

COMPARISONS OF DIFFERENT DIGITAL COMPUTING-IN-MEMORY METHODS

Methods	BS-CIM [34]	WR-CIM [35]	IR-CIM [36]	**Proposed DBS-CIM**
Reuse scheme	No Reuse	Weight Reuse	Input Reuse	**Input Reuse**
Hardware Complexity	☹	😐	☺	☺
Flexible Access	☹	☹	☹	☺
Energy Efficiency	☹	😐	😐	☺

TABLE II

DEVICE PARAMETERS

Parameters		Description	Value
RRAM [40]	LRS	Low resistance state	60 kΩ
	HRS	High resistance state	1.25 MΩ
Selector [41]	V_{IMT}	Insulator-metal transition voltage	0.6V
	V_{MIT}	Metal-Insulator transition voltage	0.02V
	R_{MET}	Metallic resistance	40 kΩ
	R_{INS}	Insulating resistance	0.12 GΩ

- A dual-bit serial transformer acceleration architecture (DBS-CIM) based on the proposed hybrid CIM cell is introduced with a Ping-Pong dataflow for high hardware utilization in attention, along with low-toggle tuning and patch-channel zero skip schemes to reduce static and dynamic matrix computing power, respectively.
- Evaluations on transformer models with different attention mechanisms show that the proposed DBS-CIM architecture saves layer-wise computing energy by up to 59.9%. Compared to prior homogeneous CIM designs, the proposed DBS-CIM provides 2.0× to 3.3× end-to-end energy efficiency. Compared to existing heterogeneous CIM designs, the proposed DBS-CIM achieves a 2.13× improvement in computational density.

II. PROPOSED DBS-CIM MACRO

This section introduces the high-density DBS-CIM macro based on differential RRAM storage, featuring flexible row-column selection and supporting efficient dual-bit serialized weight stream in input stationary CIM, as shown in Fig. 3(a). Section II-A provides an overview of the digital CIM macro, including its basic structure and inference flow. Section II-B details the specific operations of proposed DBS-CIM cell.

A. Overview of Proposed DBS-CIM Macro

The proposed DBS-CIM macro includes row and column decoders for row and column selection, input multiplier for pre-computation, LUTs for calculating the input-weight products, local adder trees and shift and adder units for summation, and 16 × 16 subarrays for storing static or dynamic weights. Each subarray (or local group) contains 8 × 8 DBS-CIM cells, whose structure is depicted in Fig. 3(b). The proposed DBS-CIM cell consists of cross-coupled inverters (P_1-P_4 and N_1-N_2), a column select transistor (N_4), a row select transistor (N_5), a reset transistor (N_3), SRAM stability transistors (N_6-N_7), and RRAM group select transistors (N_8-N_{11}). Each DBS-CIM cell incorporates two pairs of differential single-level RRAM groups with selector devices, accounting for total four groups. Each group contains 12 RRAM cells. The paired RRAM groups store the complemented weights bits for high

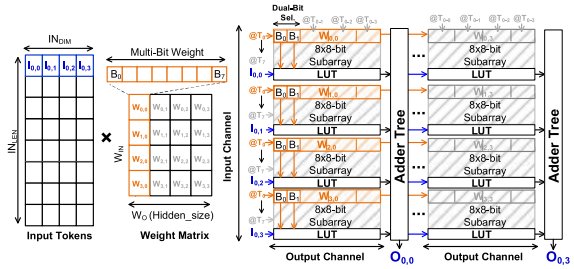


Fig. 4. Matrix multiplication mapping example of the proposed DBS-CIM macro. The vertical direction of macro maps the input channel, while the horizontal local group maps the output channel.

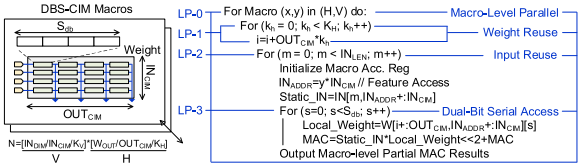


Fig. 5. The data loop of proposed DBS-CIM macro.

restore yield. Therefore, there are a total of 12 × 4 RRAMs, storing 24 bits of weights, collectively controlled by the group select (GS) signal and the RRAM select (SL) signal. The proposed DBS-CIM cell supports two-way SRAM data access. The row selection is controlled by the row-wise select signal via the row select transistor N_5, and the column selection is controlled by the column-wise select signal via the column select transistor N_4. The layout of the proposed DBS-CIM cell in 28nm logic design rule is shown in Fig. 3(c). The RRAM devices are integrated above SRAM cell, located between the fifth and sixth metal layers. The parameters of the RRAM and selector used in this work are listed in Table II.

The macro-level weight mapping method for matrix multiplication is shown in Fig. 4. The weights are evenly spread across different subarrays in macros for parallel access. Taking the 8-bit weight matrix as an example, every two bits of the weight occupy one pair of columns within a subarray, totaling eight columns for one complete 8-bit weight, which corresponds to one input channel. Weights from different input channels are mapped to different subarrays in vertical direction to enhance computational parallelism, while the weights from different output channels are located in different horizontal subarrays. Simultaneously, the 2-bit weights are read out serially to complete the input-weight multiplication.

To compute the matrix with sizes larger than a single CIM macro, the data access loop of proposed DBS-CIM is shown in Fig. 5. Assuming a single CIM macro can process IN_{CIM} input channels and OUT_{CIM} output channels simultaneously, to accommodate the matrix with sizes of IN_{DIM} input channels and W_{OUT} output channels, N macros are needed, where N is defined in the bottom of Fig. 5. The K_V (K_H) stands for the folded input (output) channels stored in the local group which equals $1 + K_{V0}(1 + K_{H0})$. K_{V0} and K_{H0} are the folding factor for input and output channels, respectively. Both K_{V0} and K_{H0} are no more than 7 and satisfy the relation of $K_{V0} + K_{H0} = 7$ for the local group with 8 × 8-bit size for storing 8-bit quantized weights. To maximize the computing parallelism between different CIM macros, K_{V0} is set as 0 for

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:26:46 UTC from IEEE Xplore. Restrictions apply.

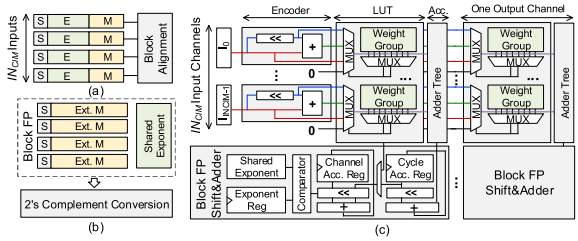


Fig. 6. The block floating point processing flow of proposed DBS-CIM macro. (a) Block alignment. (b) 2's complement conversion. (c) FP-INT process with block FP shift and adder units.

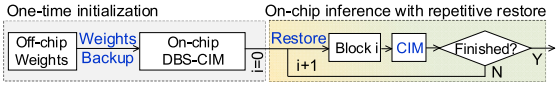


Fig. 7. Inference flow of proposed DBS-CIM.

static matrix multiplication, which means the input channels are fully unfolded across macros for parallelly processing. The data loop LP-0 is applied to all the N CIM macros to achieve macro-level parallel processing. LP-1 fetches the weights from one set of output channels to traverse all the input tokens to maximize the weight reuse. LP-2 shares input tokens among the different macros and output channels with the same input channels. LP-3 calculate the matrix multiplications in each macro with dual-bit-serial granularity, where the weights are split in 2-bit group and accessed in S_{db} cycles. For example, S_{db} is 4 with 8-bit weights. Note that the K_V loop between LP-2 and LP-3 are not shown for simplicity.

The proposed DBS-CIM macro supports FP-INT processing for accuracy-sensitive vision transformer models. The inputs with high dynamic range are quantized as FP8 to preserve the critical information during inference, while the less sensitive weights are quantized as unsigned integers with bias. To support FP computing, the block FP format is used in this work as shown in Figs. 6(a) and (b). The IN_{CIM} inputs are arranged as a block to align their exponents with additional 3-bit mantissa extension for enabling the accumulation within macro. The extended mantissa, hidden bit, and sign are extracted as 8-bit fixed point using 2's complement conversion. Next, the 2's complemented inputs are sent to the CIM macros for the DBS-CIM flow as shown in Fig. 6(c). The input I_x is firstly pre-encoded as four pre-computed results of $0 \times I_x$, $1 \times I_x$, $2 \times I_x$, and $3 \times I_x$ (standing for the multiplication results with unsigned '00', '01', '10', and '11', respectively). The pre-computed results with least significant bit truncation are shared through the output channels of DBS-CIM macro, which helps to enhance hardware reuse and reduce the depth of the adder tree. In each output channel, there are LUTs composed of local weight group and multiplexers. The fetched dual-bit weights from local weight group serially select the pre-computed results through multiplexers and send them to the adder tree for accumulations. The accumulation results are sent to the block FP shift and adder unit which composed of the cycle accumulator and channel accumulator. The cycle accumulator is used for cyclic weighting in DBS-CIM flow, while the channel accumulator is used for inter-macro summation between the split input channels.

The proposed DBS-CIM cell supports data backup, SRAM read/write, and data restore operations, as shown in Fig. 3(d).

TABLE III
SIGNAL SETTINGS OF PROPOSED DBS-CIM CELL

Operations	GS_i*	SL_i*	RSL	CSL	STL	RBL/RBLB	RST
Backup '0'	V_{DDH}	GND	GND	GND	V_{DDH}	V_{DD}/V_{DD}	GND
Backup '1'	V_{DDH}	V_{DDH}	GND	GND	V_{DDH}	GND/GND	GND
SRAM Write '0'	GND	V_{DDM}	V_{DD}	V_{DD}	V_{DD}	GND/V_{DD}	V_{DD}
SRAM Write '1'	GND	V_{DDM}	V_{DD}	V_{DD}	V_{DD}	V_{DD}/GND	GND
SRAM Read	GND	V_{DDM}	V_{DD}	V_{DD}	V_{DD}	GND/GND	GND
Restore	GND	V_{DDM}	V_{DD}	V_{DD}	V_{DD}	V_{DD}/V_{DD}	V_{DD}
	V_{DD}	GND	GND	GND	GND	GND/GND	GND

#: V_{DD}=0.9V, V_{DDH}=1.5V, V_{DDM}=0.5V;
*: GS_i and SL_i: Signals controlling the selected RRAMs.

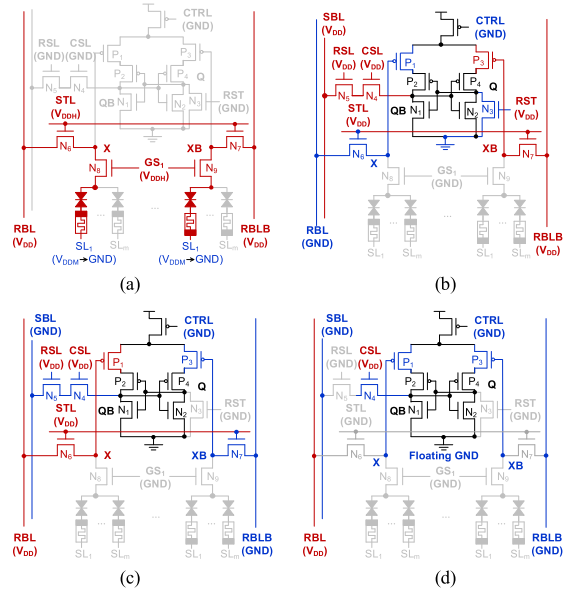


Fig. 8. Data backup and SRAM write operations of proposed DBS-CIM cell. (a) Backup '0'. (b) Write '0' into SRAM. (c) Write '1' into SRAM. (d) The state of unselected column-wise SRAM during SRAM writing.

The data backup operation is used to store weights in RRAMs. The specific control signals for these operations are listed in Table III. Thanks to the integration of high-density RRAMs, the proposed DBS-CIM is capable of accommodating large volume of weights on-chip for inference. The on-chip inference process of the proposed DBS-CIM architecture is shown in Fig. 7. Before inference, weights need to be initialized on-chip via data backup operations. During inference, the proposed DBS-CIM array only needs to repeatedly perform restore operations to call back the weights to complete computations across different transformer blocks without the need to frequently access the off-chip weights in previous CIM with limited capacity.

B. Operations of Proposed DBS-CIM Cell

The three types of basic operations (data backup, SRAM read/write operations, and data restore) supported by the proposed DBS-CIM cell are detailed in this section. The data backup operation is basically differential RRAM writing, as shown in Fig. 8(a). To write a '0' into the differential RRAM, both RBL and RBLB are driven to V_{DD}. STL is driven to V_{DDH} to turn on the SRAM stability transistors N_6 and N_7. The paired differential RRAM group select signal GS is also activated. The RRAM select signal SL transitions from V_{DDM}

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:26:46 UTC from IEEE Xplore. Restrictions apply.

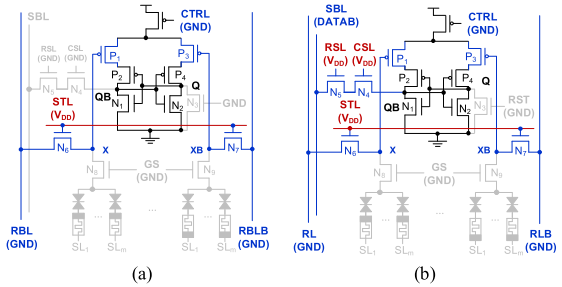


Fig. 9. (a) Data retention and (b) read operations of proposed DBS-CIM cell.

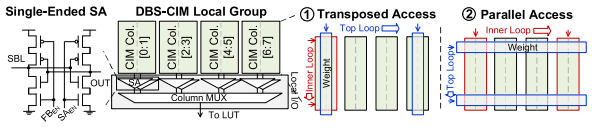


Fig. 10. Local group weight access patterns for CIM operations.

to GND to turn on the selector connected to the selected RRAM. Note that the complementary RRAM groups have reversed top-bottom electrode connections for ease of writing differential data at one time. When the write paths N_6-N_8 and N_7-N_9 are formed, the intended differential logic states can be directly written into the differential RRAM groups. With the similar method, to backup '1', the voltages of RBL/RBLB and SL only need to be reversed and the other operations are the same as the '0' case.

The single port SRAM write operations for '0' and '1' are shown in Figs. 8(b) and (c), respectively. When writing a '0', both RBLB and SBL are driven to V_{DD}. XB is driven high by RBLB to reduce the contention effect during SRAM write. The RST signal is activated high to reset the storage node Q to GND, while SBL charges QB, collectively completing the write '0' operation. When writing a '1', RBL is driven to V_{DD} to reduce the contention on the pull-up path. Simultaneously, SBL is driven to GND to write a '0' into QB, ultimately generating a logic state '1' at node Q. It should be noted that before RBL is driven to V_{DD}, the STL signal for other unselected SRAM cells unintended for writing should be deactivated, using X and XB with floating GND to sustain the cross-coupled storage nodes, as shown in Fig. 8(d).

The normal data retention state is shown in Fig. 9(a). All the RRAM group select transistors are deactivated, while the stable line STL is activated with RBL and RBLB connected to GND to enable the cross-coupled inverters. The SRAM read operation is shown in Fig. 9(b). The row and column access transistors are activated to read out the intended weights. The single-ended sense amplifier senses the bitline voltage and drives it to full swing after enabling the feedback signal FB_{EN}, as shown on the left side of Fig. 10. The SRAM read operation is used to fetch the dual-bit serialized weights for CIM. The proposed DBS-CIM macro supports two types of access patterns. The first type stores weights in a transposed manner, where the dual-bit-serial weights are aligned along the column of local group. The access starts from column to row. This pattern is favored for vector-matrix multiplication task as the input vector slice is fetched for only once with low weight reuse rate (e.g. the decoder stage in transformer with a low batch of input token in edge inference [42]). Therefore,

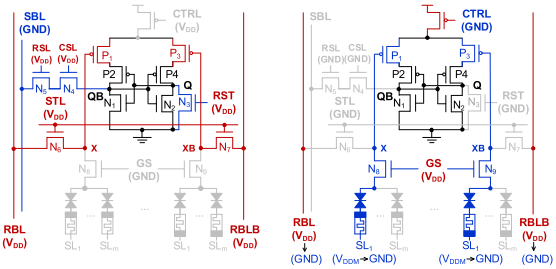


Fig. 11. Restore operations of proposed DBS-CIM cell. (a) Restore phase-1. (b) Restore phase-2.

the dual-bit data from one weight are serially readout from the same pair of bitlines to leverage the inner bit correlations enabled by the toggle tuning methods introduced in Section III-B which lead to a lower bitline flip rate and lower weight access energy. The second type performs row-wise storage and column-wise access using a feedback single-ended sense amplifier, favoring matrix-matrix multiplication task by reusing weights across multiple input vector slices to reduce the access energy.

The restore operation is used to recall the logic data stored in the differential RRAMs into the SRAM cells. The detailed operation is shown in Fig. 11. The restore process is divided into two phases. In the first phase, the control transistor supplying V_{DD} to the SRAM is turned off. Storage nodes QB and Q are reset to '0' through N_3-N_5. Simultaneously, RBL and RBLB are used to precharge nodes X and XB through N_6-N_7. In the second phase, N_3-N_7 are turned off, and the differential RRAM group select transistors are turned on. SL transitions from V_{DDM} to GND to enable the selector for the target RRAM, allowing X and XB to discharge through the differential RRAM pair. After a discharge delay, the control transistor is turned on, and V_{DD} simultaneously charges QB and Q. The discharge difference between X and XB results in different charging rates for Q and QB. When the RRAM stores a high-resistance state (HRS) and the complementary RRAM stores a low-resistance state (LRS), node X remains at a relatively high voltage, leading to a slower charging rate for QB. After the control transistor is enabled and the stable transistors are turned on, with RBL and RBLB set to GND, Q is restored to '1' and QB is restored to '0'. Conversely, Q is restored to '0' and QB is restored to '1' if the RRAM stores LRS while the complementary RRAM stores HRS.

III. Ping-Pong Dataflow and Efficient Serialized Weight Stream Computing

To efficiently support the static and dynamic matrix multiplications in transformer, this section proposes a Ping-Pong dataflow suitable for the DBS-CIM architecture with high hardware utilization. Besides, the low-toggle tuning method and computing skip mechanism are also proposed to enhance the computing energy efficiency in transformer. Section III-A introduces the Ping-Pong dataflow designed to support both static and dynamic matrix computations. Section III-B presents the computational strategies employed to optimize both static and dynamic computing processes.

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:26:46 UTC from IEEE Xplore. Restrictions apply.

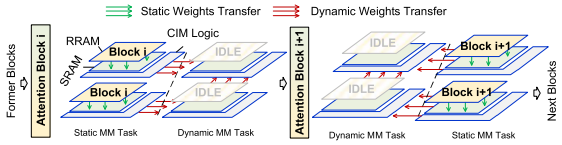


Fig. 12. The basic concept of proposed Ping-Pong computing flow. The bottom silicon SRAM and computing logic can be shared among different attention blocks to maximize the hardware utilization.

A. The Proposed Ping-Pong Dataflow

To enhance the hardware utilization of proposed heterogeneous CIM, a Ping-Pong dataflow is proposed to efficiently supports both static and dynamic computations in attention mechanism. The basic concept of proposed method is shown in Fig. 12. As the proposed DBS-CIM integrates both RRAM and SRAM in the same macro, the computing logic in the macro can be shared for both types of memory devices. For static matrix multiplication task (QKV generation and linear layers), the top RRAMs are activated by restore operation to recall weights to bottom SRAMs for computing. For dynamic matrix multiplication task (attention score computing), the top RRAMs are not used as it is not favored for write operation while it does not necessarily mean that the CIM macro must be in idleness. The bottom SRAMs can also be directly written with dynamic weights by SRAM write operation for computing. As the proposed DBS-CIM architecture stores transformer models fully on-chip and executes transformer inference block-by-block, there are inactivated blocks and CIM macros for their own static tasks. These macros are instead used for the dynamic task in current computing block to enhance the overall effective throughput. In this manner, the static and dynamic task allocations are Ping-Ponged between different DBS-CIM macros.

The proposed Ping-Pong dataflow example for ViT-B is shown in Fig. 13. In each ViT-B attention block, the input tokens are multiplied with the static weight matrix with 12 attention heads. Based on either linear attention (LA) or non-linear attention (NLA) mechanisms, the generated dynamic QKV weights are used to compute attention scores as shown in Fig. 13(a). For ViT-B, the static 8-bit $W_q/W_k/W_v$ matrixes have dimensions of 768 input channels and 768 output channels, respectively. They occupy a total of three CIM groups where each group is composed of $48 \times 6.128 \times 128$-bit DBS-CIM macros as shown in Fig. 13(b). Additional CIM group is used for temporally storing and computing the dynamic QKV weights, which allows a maximum of 256 tokens. For non-linear attention, the K matrix is transposed and stored in DBS-CIM macros while the Q matrix is read from DBS-CIM macros as the inputs for the QK^T computation. The transposed K matrix can be easily accessed by utilizing the property of the two-way select scheme of proposed DBS-CIM cell and setting the input channel folding factor K_{V0} as 7, so that the k^T vector from 12 heads can be stored along the vertical local groups. The calculated QK^T then can be sent to the DBS-CIM macros storing V for the following computing. Note that for the linear attention computing, V is read and sent to CIM macros of K^T. The calculated K^TV is sent to Q for the following computing. The static and dynamic computing are allocated between different CIM groups across attention blocks, as shown in Fig. 13(c). For block-0 computing, the CIM groups #0, #1, and #2 are used for static task, while the CIM group #3 is used for dynamic task. For block-1, the CIM groups #1, #2, and #3 are served for static task, while the CIM group #0 is used for dynamic task. The rotation helps to amortize the static weight storage in each macro and is continued until all the 12 attention blocks in ViT-B are completed. For models whose computational dimensions vary between attention blocks, the rotation mapping does not result in compact static weight storage. Therefore, the static weights are just sequentially mapped macro by macro, as in the case of [24], while the SMM and DMM CIM group configuration and the computational flow remain similar to those used in ViT-B. To further reduce dynamic weight loading energy overhead, the previous attention reformulation and pipelining techniques [43] which are also orthogonal to the proposed DBS-CIM can be directly applied to the proposed design in the future work.

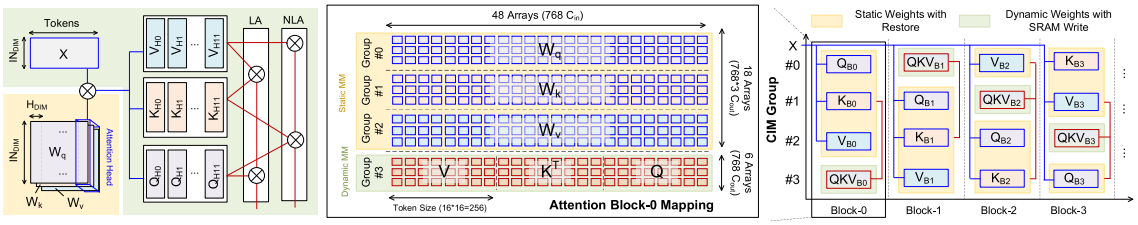
B. Serialized Weight Stream With Low-Toggle Tuning and Zero Skipping for Efficient Attention Computing

To enhance the attention computing energy efficiency, a low-toggle tuning algorithm for static weights and a zero-skipping strategy leveraging the two-way select property of proposed DBS-CIM cell are proposed for the DBS-CIM architecture. The low-toggle tuning algorithm reduces the dynamic power of SMM, while the zero-skipping strategy aims to efficiently handle the highly sparse DMM in linear attention.

1) Low-Toggle Tuning Algorithm: Since the computing power of digital CIM is closely related to the data toggle rate, reducing the toggle rate can effectively lower dynamic power. As shown in Fig. 14(a), a small change in weight value from 83 to 85 will lead to flip reduction in the proposed dual-bit-serial CIM scheme based on LUT. A further dual-bit toggle/flip rate investigation across the dynamic range for a given quantization bit width is shown in Fig. 14(b). It can be observed that the dual-bit toggle rate exhibits a distinct step-like distribution. The weights can be rounded into a nearby low-flip cluster for toggle reduction with very small value deviations or errors, which is suitable for the network inference with error tolerance. Besides, toggle rate distribution patterns are symmetric across value ranges, which enables weight rounding across the entire quantized value range for a given bit width.

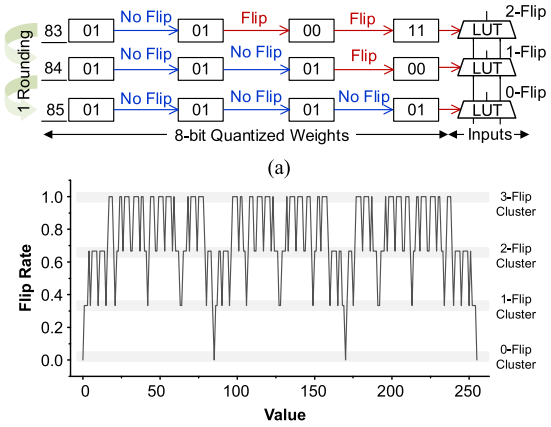
To enhance the CIM energy efficiency with low toggle rate, a post-training weight tuning algorithm is proposed, as detailed in Algorithm 1. First, the per-two-bit toggle rates for all numbers within the specified dynamic range are statistically analyzed, and the numbers are clustered based on their toggle rates. Then, for a given acceptable rounding distance, the weights are rounded via LUT under distance constraint. To mitigate the impact of errors introduced by weight rounding and enhance the adaptability to different transformer models, a layer-wise hybrid rounding search algorithm is proposed as described in Algorithm 2. First, given an overall accuracy tolerance error ($Loss_{th}$), a per-layer tolerance error ($LayerLoss_{th}$) is derived based on the parameter portion of network layers. Then, for the static weights of each layer, all possible rounding distances are iterated over, and inference is performed. If

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:26:46 UTC from IEEE Xplore. Restrictions apply.



(a) (b) (c)

Fig. 13. The proposed Ping-Pong dataflow for efficient transformer acceleration. (a) The dynamic and static matrix multiplication task in transformer models. (b) Mapping example of attention block-0 in ViT-B for ImageNet. (c) The Ping-Pong mapping for block-wise execution with high hardware utilization.



(a)

(b)

Fig. 14. (a) The toggle case in proposed input reuse based digital CIM. (b) The flip rate of numbers in 8-bit quantized dynamic value range.

Algorithm 1 Rounding LUT Construction

Input: Toggle Clusters TC, D_{max}, Value Range VR
Output: Rounding LUT

Initialize LUT for all values in VR
for each number in VR:
  Find its cluster index from TC
  if cluster index < 1:
    LUT[number] = number
  else:
    if cluster index ⩾ 2:
      candidate = find closest in TC[index-2]
      if |candidate-number| ⩽ D_{max}: // Distance Check
        LUT[number] = candidate
      else:
        Try TC[index-1] with Distance Check
        If no candidate found: LUT[number] = number
    else:
      Try TC[index-1] with Distance Check
      If no candidate found: LUT[number] = number
Return LUT

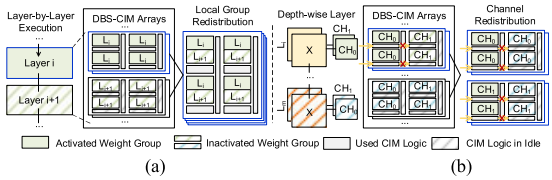
the resulting accuracy loss is within the tolerance range, the rounding distance is accepted; otherwise, the search continues for the next possible rounding distance. Once a suitable rounding value is found, the weights for that layer are fixed. The search proceeds to the next layer. This method effectively reduces the dual-bit toggle rate in DBS-CIM, thereby lowering dynamic power with minimal error. Note that the proposed weight tuning method is executed offline, ensuring that the power efficiency gains are achieved without incurring any extra hardware.

2) Static Weight Redistribution: To further enhance the hardware utilization during inference, the weight redistribution

Algorithm 2 Layer-Wise Rounding Distance Search

Input: Models and Datasets, Accuracy Loss Threshold $Loss_{th}$
Output: Layer-wise Rounding Distance Config. Table

Initialize Rounding Distance D_{max} for each layer to zero
for each layer in Model:
  Calculate the layer-wise Accuracy Loss $LayerLoss_{th}$ according to $Loss_{th}$ and parameter portion
  for each D_{max} in [4, 3, 2, 1, 0]:
    Tuning weights by D_{max} with other layer fixed and get inference accuracy Acc.
    If Accuracy Loss ($Acc_{baseline} - Acc$) < $LayerLoss_{th}$:
      Add the D_{max} to Rounding Distance Config. Table
      Break
Return Rounding Distance Config. Table



(a) (b)

Fig. 15. The proposed weight redistribution method. (a) Local group redistribution. (b) Channel redistribution.

method is proposed to fully leverage the idle CIM logic, as shown in Fig. 15. The local group redistribution mechanism swaps partial static weights between currently executing layers and adjacent layers, or layers with identical dimensions within the local group, as shown in Fig. 15(a). The original idle CIM logic used for other layers can be used to accelerate the current layer. The linear attention transformer models contain depth-wise convolution layers where the inputs are not shared across output channels. Therefore, the output channel parallelism in CIM macro cannot be utilized, which will lead to long input latency. The channel redistribution method is used to redistribute the output channels of depth-wise layers, as shown in Fig. 15(b). The output channels of different depth-wise layers are exchanged, making the computing of a single depth-wise layer distributed across multiple macros. As different CIM macros have independent input ports, it enables parallel processing of different output channels of depth-wise layers.

3) Zero Skipping for Attention Computing: Due to the use of FP alignment and ReLU activation function in attention, the dynamic Q and K matrixes exhibit natural bit-level sparsity. The leading zero sparsity of the dynamic weight matrix across all the attention layers for different attention mechanisms is shown in Fig. 16(a), where the dynamic weights are quantized as FP8 with array-level alignment. The leading zero sparsity is defined as the ratio of the count of leading zeros to the total

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:26:46 UTC from IEEE Xplore. Restrictions apply.

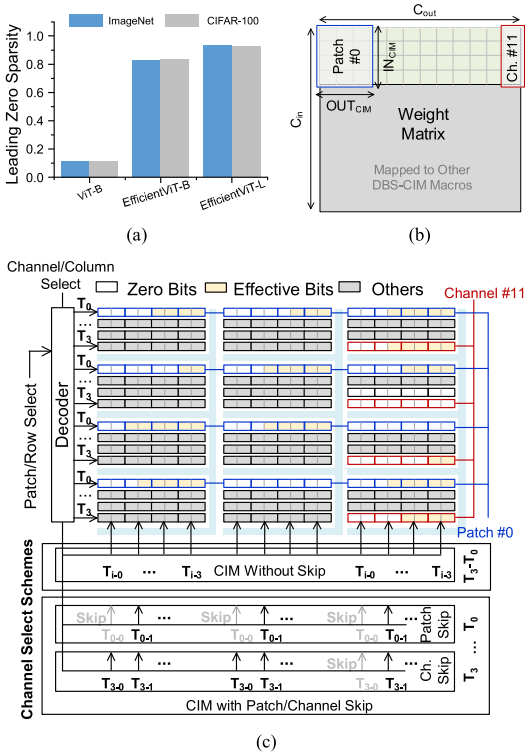


Fig. 16. The patch-channel skip mechanism of proposed DBS-CIM. (a) The leading zero sparsity of the *QKV* attention layers in transformer models with different attention mechanisms. (b) The patch and channel definition in proposed DBS-CIM architecture. (c) The compute select scheme with patch-channel skip mechanism.

number of bits in the extended mantissa. It can be observed that the non-linear transformer model has limited inherent sparsity at the bit level. The linear transformer models exhibit high sparsity above 80% by introducing ReLU function, which guarantees a high chance to skip unnecessary computations. The proposed DBS-CIM macro can leverage this sparsity characteristic of attention computing to skip in-array zero-value patch-channel computations. The patch-channel definition of the dynamic weight matrix is shown in Fig. 16(b). Each dynamic weight matrix is partitioned into several patches according to the input and output channel dimensions of CIM macro. Each patch is composed of OUT_{CIM} channels. Each channel fits into the input channel size (IN_{CIM}) of DBS-CIM macro. The weights within each patch are mapped to the local groups distributed across the whole CIM macro, while the channel-wise weights only occupy one column of local groups. The proposed DBS-CIM macro achieves skip by utilizing the two-way selection property of proposed DBS-CIM cell. The simplified dynamic weight mapping and skipping cases are shown in Fig. 16(c). During computation, if all activated weight bits in macro (patch-level sparsity) in the current cycle are detected to be zero, the computation for that cycle is skipped, like the patch skip case of patch-0. If the activated weight bits in certain output channel (channel-level sparsity) have shared zero bits, the computation for that channel is skipped, like the channel skip case of channel-11. The skip mechanism is enabled by generating a computation mask when dual-bit data written to the same row address (or across column-wise local groups) are all zero.

TABLE IV
MODELS AND DATASETS WITH SCALED SIZES FOR CLASSIFYING TASK

Models	Datasets	Data Volume
EfficientViT-Base (EViT-B)	CIFAR-100 (100 Classes) ImageNet (1000 Classes)	48.67Mb
EfficientViT-Large (EViT-L)		424.97Mb
ViT-Base (ViT-B)		679.48Mb

IV. RESULTS AND ANALYSIS

The proposed DBS-CIM cell and architecture are evaluated in this section and compared with the conventional CIM baselines. The models and datasets evaluated in this work are listed in Tables IV. This section is arranged as follows. Section IV-A introduces the experimental setup for algorithm and hardware. Section IV-B evaluates the performance of proposed DBS-CIM cell. The performance of proposed DBS-CIM architecture is evaluated and compared with state-of-the-art CIM accelerators in Section IV-C.

A. Evaluation Setup

1) Algorithm Settings: Three typical transformer models, namely ViT-Base (ViT-B) [2], EfficientViT-Base (EViT-B), and EfficientViT-Large (EViT-L) [3] with different parameter sizes and attention mechanisms are used to evaluate the proposed DBS- CIM architecture. The pretrained weights of these models are retrained on CIFAR-100 and ImageNet. The activation is quantized in FP8 format with 4-bit exponent and 3-bit mantissa bits (E4M3) and aligned with a block granularity of 16. The weights are quantized as 8-bit integer with output extended to FP32 for these three models to retain accuracy. Especially, the weights of sensitive convolution layers (such as MBConv layers) in EViT model series are preserved in 16-bit precision to maintain accuracy.

2) Hardware Settings: The macro-level evaluation of the proposed DBS-CIM is based on the post-layout results under 28nm CMOS with the physics-based Verilog-A RRAM [37] and selector [38] models, using HSPICE with 0.9V normal supply voltage at 25°C. The proposed DBS-CIM macro works at a frequency of 200 MHz. The layout of proposed DBS-CIM macro is shown in Fig. 17. The area and power breakdowns are shown in Fig. 18. The power is evaluated with the real weights and activations from transformer models. The DRAM energy is adopted from [39]. The on-chip buffer is synthesized with Memory Compiler. The area overhead induced by the buffer to the three models evaluated in this work is from 2.3% to 13.1% with the energy consumption less than 3% in the end-to-end inference. Three digital CIM baseline designs are reimplemented to evaluate the efficiency of digital computing hardware. To fairly compare the metrics, we align the performance of each baseline with the proposed design, where each digital CIM scheme provides the same number of operations per cycle:

- Baseline-1: Conventional classic bit-serial digital CIM (BS-CIM) scheme without reuse mechanism [34].
- Baseline-2: Previous weight-reuse digital CIM (WR-CIM) scheme with two-bit reuse rate [35].
- Baseline-3: Previous coalesced input-reuse digital CIM (IR-CIM) scheme [36].

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:26:46 UTC from IEEE Xplore. Restrictions apply.

TABLE V

COMPARISONS BETWEEN PROPOSED DBS-CIM CELL AND PREVIOUSLY PUBLISHED SL-NVSRAM-CIM AND SRAM CELLS IN 28NM TECHNOLOGY

Design Metrics	Standard 6T SRAM Cell	SL-nvSRAM-CIM Cell [24]	**Proposed DBS-CIM Cell**
Store Energy (fJ)	-	360	**204.3**
Restore Energy (fJ)	-	15.6	**6.68**
Bits per Cell	1	32*	**24***
Cell Layout Area (μm^2)	0.75	2.33	**1.87**
Storage Density (bit/μm^2)	1.33	13.73	**12.83**

*: Bits stored in RRAMs in each *in-situ* hybrid CIM cell.

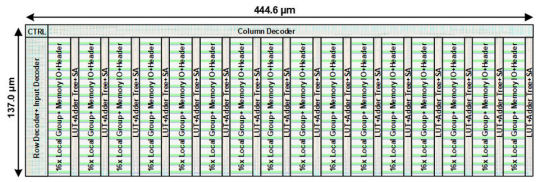


Fig. 17. The layout of proposed DBS-CIM macro.

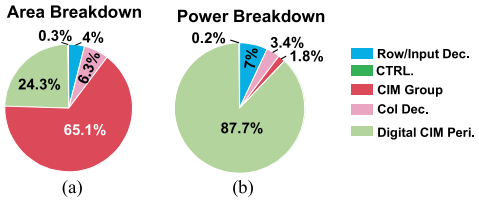


Fig. 18. (a) Area and (b) power breakdowns of proposed DBS-CIM macro.

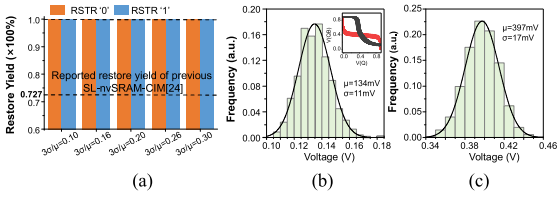


Fig. 19. Monte Carlo simulation results of proposed DBS-CIM cell at 25 °C room temperature and TT corner. (a) Restore yield under CMOS and RRAM variations. (b) The single port read static noise margin with butterfly curves. (c) The single port combined word-line margin.

The proposed design is also compared with conventional digital homogeneous CIM (RRAM-CIM and SRAM-CIM). The digital RRAM-CIM follows the design from [29], while the SRAM-CIM cell is basically the 6T SRAM cell. Both RRAM-CIM and SRAM-CIM are arranged in local CIM group like the proposed DBS-CIM. These CIM baselines are all based on conventional BS-CIM methods to validate the advantages of using different types of memory devices and CIM flows. Two typical conventional heterogeneous CIM architectures with different execution flows (layer-by-layer and layer-pipelined) based on aforementioned isolated RRAM-CIM and SRAM-CIM arrays are also compared with proposed design to demonstrate the benefits of proposed DBS-CIM architecture in area and computational density.

B. Performance of Proposed DBS-CIM Cell

1) Energy Consumption: The energy evaluations of proposed DBS-CIM cell are listed in Table V. The proposed DBS-CIM cell reduces store/restore energy by 43.3%/57.2% compared to previous SL-nvSRAM-CIM [24], thanks to the use of selector which eliminates the potential DC paths in previous voltage- divider-based selection scheme. While RRAM programming during the data backup phase remains energy-intensive, this one-time procedure has a negligible impact on overall inference power consumption.

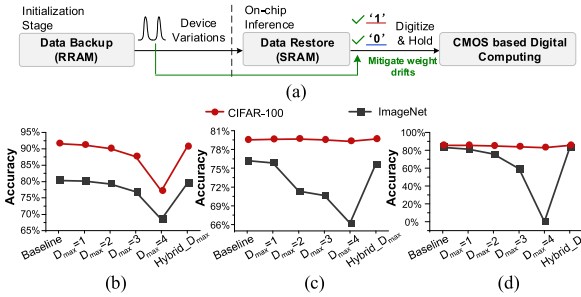


Fig. 20. The accuracy of different transformer models on CIFAR-100 and ImageNet using proposed weight toggle tuning methods. D_{max} is the tuning distance. (a) RRAM noise impact is considered as bit-flip rate determined by the restore yield during evaluation. (b) ViT-B. (c) EViT-B. (d) EViT-L.

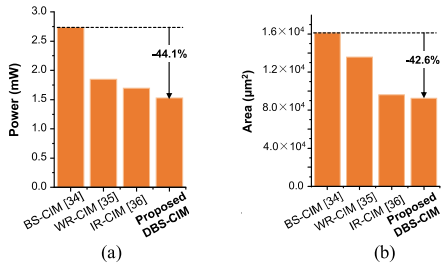


Fig. 21. The synthesized (a) area and (b) power comparisons of different digital CIM methods based on Design Compiler.

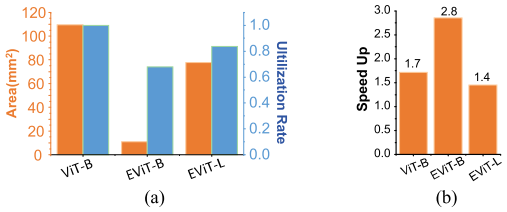


Fig. 22. (a) The area and hardware utilization of proposed DBS-CIM to fully accommodate different transformer models based on the post-layout evaluation. (b) The speed up of proposed DBS-CIM with weight redistribution compared the case without weight redistribution.

2) Storage Density: The storage density and cell area evaluation results are listed in Table V. The cell size of proposed DBS-CIM cell is evaluated as 1.87 μm^2. With 24 pairs of single-level RRAM in one cell, the storage density of proposed DBS-CIM is calculated as 12.83 bit/μm^2. Consequently, despite the increased cell size, the storage density of proposed DBS-CIM cell is enhanced by *in-situ* integration of RRAM to achieve larger on-chip capacity and reduced off-chip data access compared to the 6T SRAM cell. Compared with previous SL-nvSRAM-CIM cell, the storage density is relatively lower. This is due to the differential storage scheme adopted in the proposed DBS-CIM cell which uses two single-level RRAMs to store one data bit. This scheme trades density for better restore yield which is beneficial for applications with high robustness acquirements.

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:26:46 UTC from IEEE Xplore. Restrictions apply.

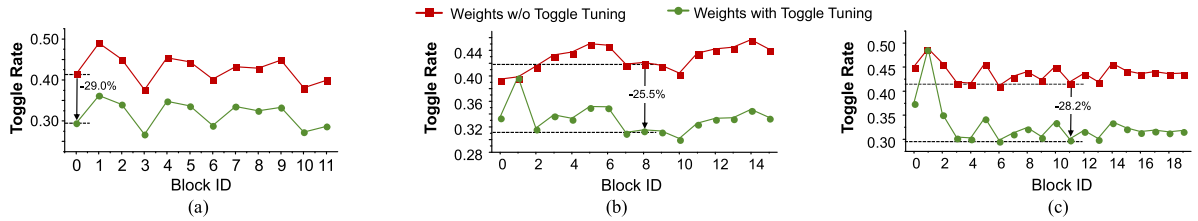


Fig. 23. The weight toggle rate of different transformer models on CIFAR-100 under the dual-bit-serial CIM scheme. (a) ViT-B. (b) EViT-B. (c) EViT-L.

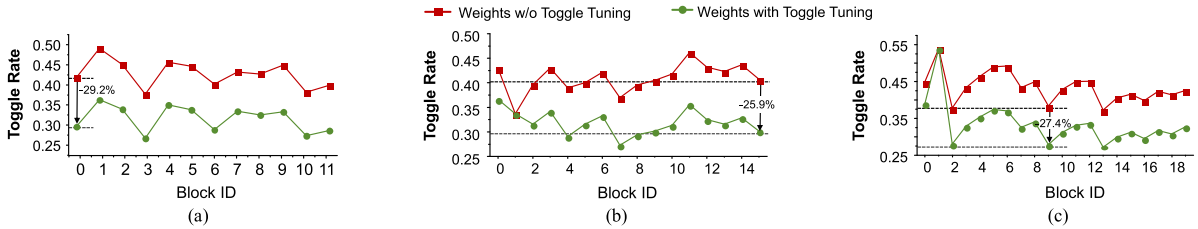


Fig. 24. The weight toggle rate of different transformer models on ImageNet under the dual-bit-serial CIM scheme. (a) ViT-B. (b) EViT-B. (c) EViT-L.

TABLE VI
COMPARISON WITH STATE-OF-THE-ART CIM ACCELERATORS

Related Works		JSSC'22 [12]	VLSI'25 [36]	TCAS-I'25 [27]	TVLSI'23 [25]	TCAS-I'23 [24]	**This Work**
CIM Mode		Analog (Current)	Digital (IR)	Analog (Current)	Analog (Current)	Analog (Current)	**Digital (DBS)**
Memory Type		RRAM	SRAM	SRAM+RRAM	SRAM+RRAM	nvSRAM with nR	**SRAM with differential nSnR**
Cell Type		1T1R	6T	10T+1T1R	8T+1R	15T with 32R	**15T with 24 Differential R**
CIM Cell Size (μm^2)		0.79	-	3.7 (SRAM)	0.15† (SRAM)	2.33	**1.87**
Normal Supply Voltage (V)		0.9	1.0	0.5	0.5	0.9	**0.9**
CIM Process Node (nm)		40	28	32	32	28	**28**
Storage Density (bit/μm^2)		2.02 (cell) 0.10 (macro)	0.38 (macro)	0.27 (SRAM cell)	6.5† (SRAM cell)	13.73 (cell) 10.05 (macro)	**12.83 (cell) 6.18 (macro)**
Application		CNN	Transformer (Non-Linear Attention)	Transformer (Non-Linear Attention)	Transformer (Non-Linear Attention)	CNN	**Transformer (Linear/Non-Linear Attention)**
Precision		INT	INT	INT	INT	INT	**FP-INT**
Weight Access Pattern		Row-wise	Row-wise	Row-wise	Row-wise	Row-wise	**Row-and-Column-wise**
Frequency (MHz)		100	410	200	200	500	**200**
Data Transfer Energy across Hybrid Memory (fJ/bit)		-	-	-	-	15.6	**6.7**
Attention Support	Off-Chip Static Weight Loading	DS**	Yes	No	No	DS**	**No**
	Unified Hardware for SMM and DMM	DS**	No	No	No	DS**	**Yes**
Weight Transfer Parallelism		Row-level	Row-level	Row-level	Row-level	Array-level	**Array-level**
Normalized Energy Efficiency* (TOPS/W)		104.8	2320±	398.3	1019.7‡	195	**557.5**‖

†: No layout provided and estimated by feature sizes. *: The energy efficiency results are all normalized to input/weight of 1b/1b in 28nm technology node according to the open-source technology scaling tool [44]. **: Do not support attention computing. ±: Off-chip data transfer is not included. ‡: DC results without post-layout simulations. ‖: Post-layout simulations.

3) Robustness of Restore Yield and Noise Margin: To assess the non-linearity effect introduced by RRAM's inherent conductive filament variation on proposed DBS-CIM cell, the restore yield is evaluated and illustrated in Fig. 19(a), considering both CMOS and RRAM filament variations. The CMOS variation is considered in TT-corner, while the RRAM filament variation is defined as the filament gap distance variation and is swept from $3\sigma/\mu = 0.1$ to $3\sigma/\mu = 0.3$ for simulations. The restore yield of proposed DBS-CIM cell has no loss under high RRAM variation of $3\sigma/\mu = 0.3$ thanks to the differential restore mechanism with high sensing margin. As a comparison, the previous SL-nvSRAM-CIM only has 72.7% restore yield. Besides, the maximum allowable number of RRAM in [24] is constrained by the selection scheme, which leads to a degradation in restore yield as the number of RRAM increases. The single port read and write margin simulation results of proposed DBS-CIM cell for SRAM mode are shown in Fig. 19(b) and (c), respectively. The butterfly

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:26:46 UTC from IEEE Xplore. Restrictions apply.

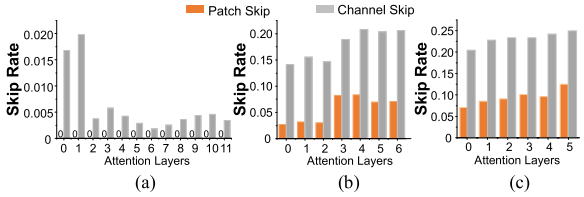


Fig. 25. The skip rate of transformer models on CIFAR-100. The layer-wise results of (a) ViT-B, (b) EViT-B, and (b) EViT-L.

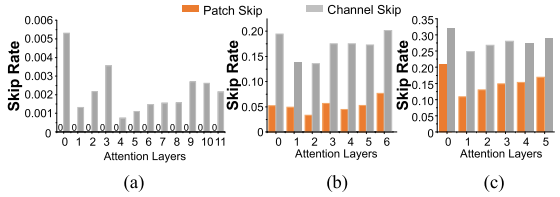


Fig. 26. The skip rate of transformer models on ImageNet. The layer-wise results of (a) ViT-B, (b) EViT-B, and (b) EViT-L.

curves for evaluating read static noise margin are also included in Fig. 19(b). The mean read and write margins are simulated as 134 mV and 397 mV, respectively, which are high enough to ensure robust SRAM read and write operations.

C. Performance of Proposed DBS-CIM Architecture

1) Tuning Algorithm and Accuracy: The accuracy evaluation results of different transformer models under different weight tuning granularities are shown in Fig. 20, where RRAM variation is considered by setting the bit-flip rate during inference which is determined by the restore yield. The accuracy drop after FP-INT quantization for the three evaluated models is 0.86% on average. The baseline accuracy is evaluated under FP-INT quantization as stated in Section IV-A. The accuracy may drop as the weight rounding distance D_{max} increases, which trades off energy efficiency against robustness. However, it can also be observed that different models have different sensitivity regarding D_{max}. For conventional transformer models with non-linear attention mechanism, the accuracy does not drop significantly until D_{max} equals 4, while the transformer models with linear attention have higher sensitivity for the change of D_{max}. Besides, increasing the classifying complexity also increases the sensitivity to D_{max} for models. To accommodate the varying accuracy requirements of different models across datasets, a hybrid D_{max} strategy is applied across model blocks, where a low D_{max} is used for sensitive blocks, while a higher D_{max} is assigned to less sensitive ones. The maximal allowable accuracy loss during D_{max} search is set below 1%. As the validated results show, the accuracy loss is well controlled with hybrid D_{max} method. The highest accuracy loss observed is only 0.91%, occurring in ViT-B on CIFAR-100, which is considered acceptable.

2) Area: The synthesized area and power evaluation results of different digital CIM peripheries based on Design Compiler are shown in Figs. 21(a) and (b), respectively. The proposed DBS-CIM achieves a 42.6% reduction in area overhead and a 44.1% reduction in power consumption relative to the previous BS-CIM. To increase the on-chip capacity under the same area, relatively low compute-storage ratio is adopted in this work, where every 8 × 8-bit DBS-CIM subarray shares one LUT computing unit. The macro-level storage density is 6.18 bits/μm^2. To accommodate models fully on-chip, multiple DBS-CIM macros are needed. The total areas for different models are shown in Fig. 22(a). The area consumption scales with model parameters. The layer form factor also impacts the total area as the weight mapping cannot fully leverage the storage resources, lowering hardware utilization rate. Overall, up to 109.5 mm^2 on-chip area is needed to store the largest model with 679.5 Mb parameters.

3) Throughput: A single DBS-CIM macro processes 8-bit input from 16 input channels with 2-bit weight from 16 output channels, which guarantees a throughput of 1.6 TOPS under 200MHz. To efficiently execute inference task, the inter macro computing parallelism is considered. As the proposed DBS-CIM is executed in block granularity, the weights are activated only for one model block at a time, leaving other CIM macros that store other model blocks idle. To improve CIM macro utilization, weight redistribution method is applied. This approach enables the originally idle DBS-CIM macros to be activated simultaneously. The overall speed up evaluations with weight redistribution are shown in Fig. 22(b), where a maximum 2.8× speedup is achieved in EViT-B inference.

4) Energy Efficiency: The impact of tuning methods on the data toggle rate is evaluated in Figs. 23 and 24. With the proposed DBS-CIM mechanism, the weight toggle rate remains around 50% across models and datasets, suggesting that the original weight pattern could lead to high dynamic power. The weight tuning method with hybrid D_{max} can achieves steady weight toggle rate reduction which helps to reduce the power consumption. The skip mechanism evaluations are shown in Figs. 25 and 26. By applying ReLU function to QK matrix, the attention mechanism exhibits high sparsity, providing skip chances to reduce the energy consumption of DMM which can occupy around 11% of the total computing energy depending on the models. Layer-wise inference energy consumptions are also evaluated in Figs. 27 and 28, for different transformer models and datasets. The results are all normalized to the conventional digital BS-CIM [34] without hardware reuse. The proposed DBS-CIM further reduces the energy consumption of previous hardware reuse methods by introducing toggle tuning and skip mechanism while maintaining inference accuracy. Given the high parallelism of restore operations for SMM and the limited dynamic parameter sizes for DMM, the SMM and DMM parameter loading process accounts for only a small portion of the total inference energy and latency in the proposed DBS-CIM architecture. Overall, the proposed DBS-CIM achieves 59.9% and 15% energy reduction compared to previous BS-CIM [34] and IR-CIM [36], respectively.

5) Comparisons With Conventional CIM Architecture and State-of-the-Art CIM Accelerators: The comparisons of previous homogeneous CIM with proposed DBS-CIM are shown in Fig. 29. The proposed DBS-CIM attains higher energy efficiency than previous SRAM-CIM, primarily by eliminating off-chip DRAM accesses and implementing efficient CIM methods. The proposed DBS-CIM method also outperforms previous RRAM-CIM mainly due to the difference in CIM

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:26:46 UTC from IEEE Xplore. Restrictions apply.

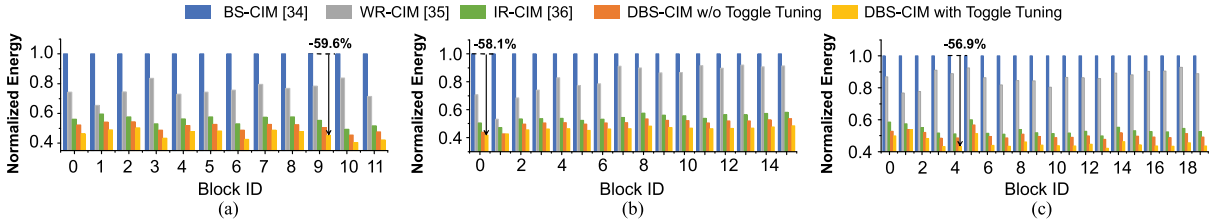


Fig. 27. The layer-wise inference energy of different transformer models on CIFAR-100 based on post-layout results. (a) ViT-B. (b) EViT-B. (c) EViT-L.

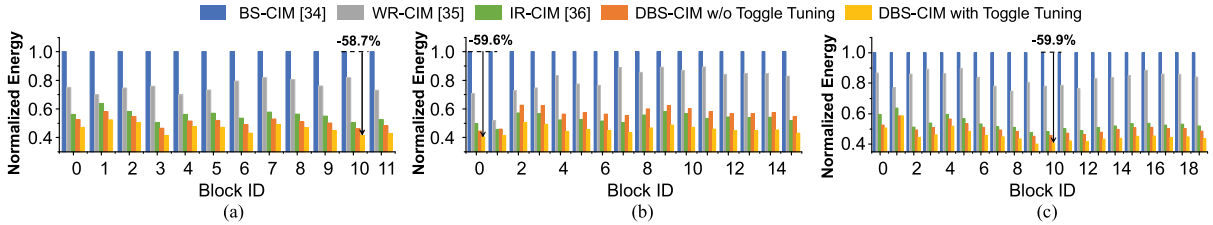


Fig. 28. The layer-wise inference energy of different transformer models on ImageNet based on post-layout results. (a) ViT-B. (b) EViT-B. (c) EViT-L.

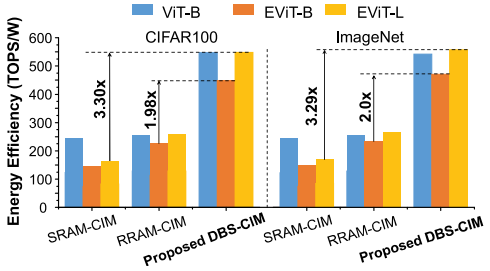


Fig. 29. The end-to-end energy efficiency between previous homogeneous CIM and proposed DBS-CIM for different transformer models on CIFAR-100 and ImageNet based on post-layout results.

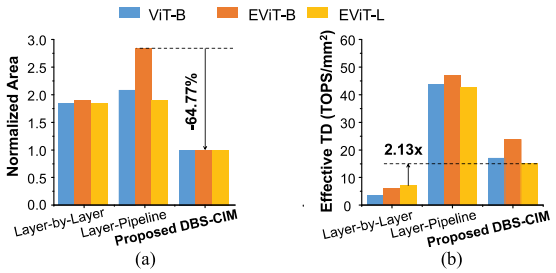


Fig. 30. (a) Area and (b) throughput density evaluations of conventional heterogeneous CIM and proposed DBS-CIM for different transformer models based on post-layout results. The areas are all normalized to DBS-CIM.

scheme. Furthermore, the proposed DBS-CIM employs SRAM to store dynamic weights in transformers, thereby avoiding the endurance issues associated with previous RRAM-CIM. Overall, the proposed DBS-CIM achieves up to 3.3× and 2.0× enhanced energy efficiency as compared with conventional SRAM-CIM and RRAM-CIM, respectively. The comparisons of proposed DBS-CIM with conventional heterogeneous CIM are shown in Fig. 30, in terms of area and effective throughput density (considering the idle CIM macros during inference). The first heterogeneous CIM uses layer-by-layer execution. It provisions SRAM-CIM for only one attention block, forcing multiple blocks to share the same SRAM-CIM arrays. This blocks other layers and leaves RRAM-CIM arrays idle due to SRAM contention, reducing effective throughput density despite low area overhead. The second type uses layer-pipelined execution with sufficient hardware for all blocks, achieving high throughput density at the cost of large area overhead. In contrast, by stacking RRAM directly over silicon to share computing logic with SRAM, DBS-CIM reduces area overhead by up to 64.77% compared to the layer-pipelined heterogeneous design as shown in Fig. 30(a). Moreover, thanks to redistribution method and Ping-Pong computing flow, the proposed DBS-CIM achieves 2.13× effective throughput density over the layer-by-layer design by reactivating otherwise-idle digital CIM circuitry as shown in Fig. 30(b).

The proposed DBS-CIM architecture also demonstrates significant advantages over existing CIM solutions, as listed in Table VI. For fair comparison with the previous heterogeneous CIM works [24], [27] which also evaluate the end-to-end inference (including off-chip data access with post-layout implementation), the energy efficiency of proposed DBS-CIM is enhanced by 1.40× to 2.85×. Beyond density and endurance benefits over previous homogeneous CIM designs [12], [36], this work also features a robust and unified hardware that flexibly supports SMM and DMM which ensures high utilization and enhanced throughput density.

V. Conclusion

This paper presents a full-digital heterogeneous CIM architecture based on hybrid RRAM and SRAM for efficient on-chip transformer acceleration. A customized high-density SRAM-CIM cell with differential RRAM storage and two-way selection is proposed to achieve high data restore yield and CIM flexibility. The input reuse based dual-bit serialized CIM scheme with low toggle-rate tuning algorithm and zero skipping scheme is proposed for the CIM macro to reduce the digital CIM peripheries complexity and both the static and dynamic matrix multiplication computing power. Besides, a Ping-Pong dataflow is specifically designed for the proposed CIM architecture to accelerate attention mechanism with high

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:26:46 UTC from IEEE Xplore. Restrictions apply.

hardware utilization. Experimental results on different transformer models show that the proposed DBS-CIM architecture reduces layer-wise computing energy by up to 59.9% compared to conventional digital CIM method. Compared to prior homogeneous CIM designs, it achieves 2.0× to 3.3× higher end-to-end energy efficiency, and outperforms existing heterogeneous CIM designs by 2.13× in computational density. The proposed DBS-CIM architecture offers a promising solution for energy-efficient inference in resource-constrained devices.

REFERENCES

[1] A. Vaswani et al., "Attention is all you need," in *Proc. Adv. Neural Inf. Process. Syst.*, 2017, pp. 6000–6010.

[2] A. Dosovitskiy, "An image is worth 16×16 words: Transformers for image recognition at scale," in *Proc. ICLR*, 2021, pp. 1–21.

[3] Y. Xie and Y. Liao, "Efficient-ViT: A light-weight classification model based on CNN and ViT," in *Proc. 6th Int. Conf. Image Graph. Process.*, Jan. 2023, pp. 64–70.

[4] S. Zhuoran, Z. Mingyuan, Z. Haiyu, Y. Shuai, and L. Hongsheng, "Efficient attention: Attention with linear complexities," in *Proc. IEEE Winter Conf. Appl. Comput. Vis. (WACV)*, Jan. 2021, pp. 3530–3538.

[5] H. Shi, H. Shao, W. Mao, and Z. Wang, "Trio-ViT: Post-training quantization and acceleration for softmax-free efficient vision transformer," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 72, no. 3, pp. 1296–1307, Mar. 2025.

[6] R. Guo et al., "CIMFormer: A systolic CIM-array-based transformer accelerator with token-pruning-aware attention reformulating and principal possibility gathering," *IEEE J. Solid-State Circuits*, vol. 59, no. 10, pp. 3317–3329, Oct. 2024.

[7] X. Fu et al., "P3 ViT: A CIM-based high-utilization architecture with dynamic pruning and two-way ping-pong macro for vision transformer," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 70, no. 12, pp. 4938–4948, Dec. 2023.

[8] X. Fu, J. Yue, M. Faizan, Z. Li, Q. Huo, and F. Zhang, "SHMT: An SRAM and HBM hybrid computing-in-memory architecture with optimized KV cache for multimodal transformer," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 72, no. 6, pp. 2712–2725, Jun. 2025.

[9] D. Han and A. P. Chandrakasan, "MEGA.Mini: A universal generative AI processor with a new big/little core architecture for NPU," in *Proc. IEEE Int. Solid-State Circuits Conf. (ISSCC)*, Feb. 2025, pp. 1–3.

[10] Z. Sun, S. Kvatinsky, X. Si, A. Mehonic, Y. Cai, and R. Huang, "A full spectrum of computing-in-memory technologies," *Nature Electron.*, vol. 6, no. 11, pp. 823–835, Nov. 2023.

[11] A. Sebastian, M. Le Gallo, R. Khaddam-Aljameh, and E. Eleftheriou, "Memory devices and applications for in-memory computing," *Nature Nanotechnol.*, vol. 15, no. 7, pp. 529–544, Mar. 2020.

[12] W. Li, X. Sun, S. Huang, H. Jiang, and S. Yu, "A 40-nm MLC-RRAM compute-in-memory macro with sparsity control, on-chip write-verify, and temperature-independent ADC references," *IEEE J. Solid-State Circuits*, vol. 57, no. 9, pp. 2868–2877, Sep. 2022.

[13] C.-Y. Chang, C.-T. Huang, Y.-C. Chuang, K.-C. Chou, and A.-Y. Wu, "BFP-CIM: Runtime energy-accuracy scalable computing-in-memory-based DNN accelerator using dynamic block-floating-point arithmetic," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 71, no. 5, pp. 2079–2092, May 2024.

[14] Z. Zhou et al., "ArPCIM: An arbitrary-precision analog computing-in-memory accelerator with unified INT/FP arithmetic," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 72, no. 5, pp. 2229–2242, May 2025.

[15] F. Zhang et al., "A 65-nm RRAM compute-in-memory macro for genome processing," *IEEE J. Solid-State Circuits*, vol. 59, no. 7, pp. 2093–2104, Jul. 2024.

[16] V. Sharma et al., "A 64 kb reconfigurable full-precision digital RRAM-based compute-in-memory for artificial intelligence applications," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 69, no. 8, pp. 3284–3296, Aug. 2022.

[17] V. Sharma, X. Zhang, N. S. Dhakad, and T. T.-H. Kim, "FlexDCIM: A 400 MHz 249.1 TOPS/W 64 Kb flexible digital compute-in-memory SRAM macro for CNN acceleration," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 72, no. 10, pp. 5696–5707, Oct. 2025.

[18] H. You, W. Li, D. Shang, Y. Zhou, and S. Qiao, "A 1–8b reconfigurable digital SRAM compute-in-memory macro for processing neural networks," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 71, no. 4, pp. 1602–1614, Apr. 2024.

[19] D. Wang, W. He, Q. Wang, H. Jiao, and Y. Sun, "Robust monolithic 3D carbon-based computing-in-SRAM with variation-aware bit-wise data-mapping for high-performance and integration density," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 72, no. 3, pp. 1229–1242, Mar. 2025.

[20] F. Tu et al., "ReDCIM: Reconfigurable digital computing-in-memory processor with unified FP/INT pipeline for cloud AI acceleration," *IEEE J. Solid-State Circuits*, vol. 58, no. 1, pp. 243–255, Jan. 2023.

[21] J. Yue et al., "An energy-efficient computing-in-memory NN processor with set-associate blockwise sparsity and ping-pong weight update," *IEEE J. Solid-State Circuits*, vol. 59, no. 5, pp. 1612–1627, May 2024.

[22] J. Song, X. Tang, X. Qiao, Y. Wang, R. Wang, and R. Huang, "A 28 nm 16 kb bit-scalable charge-domain transpose 6T SRAM in-memory computing macro," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 70, no. 5, pp. 1835–1845, May 2023.

[23] Z. Meng, Y. Sun, and W. Qian, "Write or not: Programming scheme optimization for RRAM-based neuromorphic computing," in *Proc. 59th ACM/IEEE Design Autom. Conf.*, Jul. 2022, pp. 985–990.

[24] Y. Sun et al., "CREAM: Computing in RRAM-assisted energy{-} and area-efficient SRAM for reliable neural network acceleration," *IEEE Trans. Circuits Syst. I, Regular Papers*, vol. 70, no. 8, pp. 3198–3211, Aug. 2023.

[25] S. Sridharan, J. R. Stevens, K. Roy, and A. Raghunathan, "X-Former: In-memory acceleration of transformers," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 31, no. 8, pp. 1223–1233, Aug. 2023.

[26] S. Liu et al., "HARDSEA: Hybrid analog-ReRAM clustering and digital-SRAM in-memory computing accelerator for dynamic sparse self-attention in transformer," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 32, no. 2, pp. 269–282, Feb. 2024.

[27] S. Dong et al., "Topkima-former: Low-energy, low-latency inference for transformers using top-k in-memory ADC," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 72, no. 6, pp. 2509–2519, Jun. 2025.

[28] C. Mu et al., "A 28-nm RRAM/SRAM collaborative CIM accelerator supporting RRAM-endurance-latency awareness for edge fine-tuning," *IEEE J. Solid-State Circuits*, vol. 60, no. 10, pp. 3626–3638, Oct. 2025.

[29] L. Xu, S. Yuan, D. Wang, Y. Chen, X. Li, and Y. Sun, "HEIRS: Hybrid three-dimension RRAM{-} and SRAM-CIM architecture for multi-task transformer acceleration," in *Proc. 61st ACM/IEEE Design Autom. Conf.*, Jun. 2024, pp. 1–6.

[30] S. Yuan et al., "HPIM-NoC: A priori-knowledge-based optimization framework for heterogeneous PIM-based NoCs," in *Proc. 62nd ACM/IEEE Design Autom. Conf. (DAC)*, Jun. 2025, pp. 1–7.

[31] H.-H. Hsu et al., "A 22 nm 41.8 TFLOPS/W AI-edge transformer/CNN nonvolatile-processor using QKV-softmax-layer-fused hybrid ReRAM-CIM and concurrent-transpose/non-transpose SRAM-CIM," in *Proc. Symp. VLSI Technol. Circuits (VLSI Technol. Circuits)*, Jun. 2025, pp. 1–3.

[32] J. Wang et al., "A high-density energy-efficient CNM macro using hybrid RRAM and SRAM for memory-bound applications," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 33, no. 8, pp. 2339–2343, Aug. 2025.

[33] A. R. Bommana, F. Firouzi, and K. Chakrabarty, "COMET-3D: Compute-in-memory based transformer accelerator with optimized pipeline and 3D heterogeneous integration," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 45, no. 3, pp. 1937–4151, Aug. 2025.

[34] Y.-D. Chih et al., "16.4 An 89 TOPS/W and 16.3 TOPS/mm^2 all-digital SRAM-based full-precision compute-in memory macro in 22nm for machine-learning edge applications," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Feb. 2021, pp. 252–254.

[35] C.-F. Lee et al., "A 12 nm 121-TOPS/W 41.6-TOPS/mm^2 all digital full precision SRAM-based compute-in-memory with configurable bit-width for AI edge applications," in *Proc. IEEE Symp. VLSI Technol. Circuits (VLSI Technol. Circuits)*, Jun. 2022, pp. 24–25.

[36] Y. Wang et al., "LLM-CIM: A 28 nm 126.7 TOPS/W input-LUT-based digital CIM macro with reconfigurable matrix multiplication and nonlinear operation modes for LLMs," in *Proc. Symp. VLSI Technol. Circuits (VLSI Technol. Circuits)*, Jun. 2025, pp. 1–3.

[37] P.-Y. Chen and S. Yu, "Compact modeling of RRAM devices and its applications in 1T1R and 1S1R array design," *IEEE Trans. Electron Devices*, vol. 62, no. 12, pp. 4022–4028, Dec. 2015.

[38] W.-Y. Tsai et al., "Enabling new computation paradigms with HyperFET–an emerging device," *IEEE Trans. Multi-Scale Comput. Syst.*, vol. 2, no. 1, pp. 30–48, Jan. 2016.

[39] M. Gao et al., "TETRIS: Scalable and efficient neural network acceleration with 3D memory," *ASPLOS*, vol. 45, no. 1, pp. 751–764, Apr. 2017.

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:26:46 UTC from IEEE Xplore. Restrictions apply.

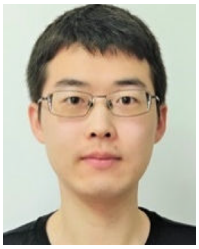
[40] M. Zhao et al., "Characterizing endurance degradation of incremental switching in analog RRAM for neuromorphic systems," in *IEDM Tech. Dig.*, Dec. 2018, pp. 20.2.1–20.2.4.
[41] N. Shukla, R. K. Ghosh, B. Grisafe, and S. Datta, "Fundamental mechanism behind volatile and non-volatile switching in metallic conducting bridge RAM," in *IEDM Tech. Dig.*, Dec. 2017, pp. 4.3.1–4.3.4.
[42] C. Li et al., "H2-LLM: Hardware-dataflow co-exploration for heterogeneous hybrid-bonding-based low-batch LLM inference," in *Proc. 52nd Annu. Int. Symp. Comput. Archit.*, Jun. 2025, pp. 194–210.
[43] R. Guo et al., "A 28-nm 28.8-TOPS/W attention-based NN processor with correlative CIM ring architecture and dataflow-reshaped digital-assisted CIM array," *IEEE J. Solid-State Circuits*, vol. 60, no. 1, pp. 332–346, Jan. 2025.
[44] S. Sarangi and B. Baas, "DeepScaleTool: A tool for the accurate estimation of technology scaling in the deep-submicron era," in *Proc. IEEE Int. Symp. Circuits Syst. (ISCAS)*, May 2021, pp. 1–5.



Dengfeng Wang (Student Member, IEEE) received the B.E. degree from the Department of Physics and Astronomy, Shanghai Jiao Tong University, Shanghai, China, in 2020. He is currently pursuing the Ph.D. degree with the Department of Micro-Nano Electronics, Shanghai Jiao Tong University. His research interests include computing-in-memory and monolithic three-dimensional circuit design.



Liukai Xu received the B.E. degree from the Department of Micro-Nano Electronics, Shanghai Jiao Tong University, Shanghai, China, in 2021, where he is currently pursuing the M.E. degree. His current research interests include computing-in-memory and neural-network accelerator design.



Shuai Yuan received the B.E. and M.S. degrees from the Department of Micro-Nano Electronics, Shanghai Jiao Tong University, Shanghai, China, in 2016 and 2022, respectively. He is currently pursuing the Ph.D. degree with the School of Electronic, Information and Electrical Engineering, Shanghai Jiao Tong University. His research interests include hardware–software co-design and computing in memory.



Weifeng He (Senior Member, IEEE) received the B.S., M.S., and Ph.D. degrees in microelectronics and solid state electronics from Harbin Institute of Technology, China, in 1999, 2001, and 2005, respectively.

He is currently a Professor with the Department of Micro-Nano Electronics, Shanghai Jiao Tong University, China. His research interests include VLSI architecture for video signal processing, reconfigurable processor architecture, and low power circuit design.



Guanghui He (Member, IEEE) received the B.S. degree in electronic engineering from the University of Electronic Science and Technology of China, Chengdu, China, in 2002, and the Ph.D. degree in electronic engineering from Tsinghua University, Beijing, China, in 2007. He is currently a Professor with the Department of Micro-Nano Electronics and the MoE Key Laboratory of Artificial Intelligence, Shanghai Jiao Tong University. His research interests include energy efficient algorithms and circuit design.



Fengbin Tu (Senior Member, IEEE) received the B.S. degree from the School of Electronic Engineering, Beijing University of Posts and Telecommunications, Beijing, China, in 2013, and the Ph.D. degree from the Institute of Microelectronics, Tsinghua University, Beijing, in 2019. He was a Post-Doctoral Fellow with the AI Chip Center for Emerging Smart Systems (ACCESS), Hong Kong, China, from 2022 to 2023. He is currently an Assistant Professor and the Associate Director with the Institute of Integrated Circuits and Systems, The Hong Kong University of Science and Technology, Hong Kong. His research interests include AI chip, computer architecture, reconfigurable computing, and computing-in-memory.



Xueqing Li (Senior Member, IEEE) received the B.S. and Ph.D. degrees from the Department of Electronic Engineering, Tsinghua University, Beijing, China, in 2007 and 2013, respectively.

He is currently an Associate Professor with the Department of Electronic Engineering, Tsinghua University. His research interests include high-performance data converter circuit design, emerging memory, and memory-oriented computing with CMOS and beyond-CMOS technologies.



Yanan Sun (Senior Member, IEEE) received the B.E. degree in microelectronics from Shanghai Jiao Tong University, Shanghai, China, in 2009, and the Ph.D. degree from The Hong Kong University of Science and Technology, Hong Kong, in 2015.

She is currently an Associate Professor with the Department of Micro-Nano Electronics, Shanghai Jiao Tong University. Her research interests include energy-efficient VLSI circuit and system design with emerging nanotechnologies. She received the Best Paper Award nomination at IEEE DATE and IEEE ICTA, as well as the Best Paper Award at IEEE ICM.

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:26:46 UTC from IEEE Xplore. Restrictions apply.